

Mission Control — build spec

The operator console. Visual: mission-control-mockup.html. Full page list: frontend-page-map.md §(a). This file drills into the **pipeline board** — the centerpiece — and how it wires to the engine we ran live.

Last updated: 2026-08-08

What it is

The desktop console where NMAO staff run and watch a round. The common case is fully automated (the engine does the work); the operator's job is to advance stages, read exceptions, and roll back when needed. Everything it triggers is an idempotent step on the engine, so buttons are safe to re-press.

The pipeline board (/control/rounds/[roundId])

A round moves through the engine's state machine; the board shows each stage, its status, the key counts, and the one action available next.

Stage (board)	Engine step	Writes	“Done” detail	Guardrail before running
Entries closed	(state closed)	—	valid entry count	—
Divide	divide	divisions, pods, seats entries	divisions / pods / seated / #collapsed	round closed, >0 valid entries
Assign judges	assign_judges	judge_assignments	assignments, shortfalls	pods exist
Judging window	(judges score in app)	judge_assignments.score	scored / total (%)	—
Resolve	resolve	results, skill_ratings, rating_history	results, ratings updated	all pods have scores (else reopen)
Distribute	distribute	medals, medal_shipments	medals (G/S/B/part.), boxes	every pod resolved
Finalize	(operator)	freezes scheme version, publishes	—	round distributed

Each stage card: status dot (done / running / next / idle / **error**), title, detail line, a Run or View action, and last-run timestamp. A stage in error is re-runnable (the claim clears on error).

How a stage runs

Buttons POST to the `round-controller` edge function: `{ roundId, step }` where step is divide / assign_judges / resolve / distribute.

- **Auth:** the function is gated. From the browser, call it with the signed-in **staff session token** (Supabase auth JWT) — the gate accepts a user whose `auth_user_id` maps to a staff row. The `sb_secret_service` path is for cron/internal only; never ship a secret key to the client.
- **Idempotent:** re-pressing a done step returns `ran:false` and changes nothing (`claim_step`). Show it as a toast, not an error.
- **Optimistic UI:** flip the dot to running on click; reconcile from the response + realtime.

Live status (realtime)

Subscribe to `round_step_runs` for this round → drives every stage dot without polling. Also subscribe to `judge_assignments` (judging %) and results (resolve progress). Loading = skeleton lanes; never a bare spinner.

Right rail

- **Exceptions & guardrails** — the engine's flags (under-floor pod that couldn't collapse, judge shortfall from own-school conflicts) surface here in red with a jump-to link; otherwise an all-clear. Below, the standing guardrails (blocked transitions) as read-only reminders.
- **Divisions & pods** — compact summary (tier-colored: Sapphire beginner, Amethyst intermediate, Ruby advanced), each with pod size + judge count.
- **Activity log / rollback** — recent `round_step_runs` and `engine_audit` entries (who/when); a Roll back to... control that clears downstream artifacts and re-runs from a chosen stage (writes an `engine_audit` row).

The rest of the console (see frontend-page-map §a)

`/control` dashboard, `/control/divisions`, `/entries`, `/judges`, `/results`, `/medals`, `/finance`, `/audit`, `/scheme` (edit + **simulate** via the divisioning core), `/settings`. The board links into these. Suggested build order: **board** → **divisions/pods** → **results** → **medals** → **judges/entries** → **scheme/simulate** → **finance** → **audit** → **settings**.

Tokens & feel

Dark “control room”: brand-tokens.md neutrals + gold for primary actions; the metallic spectrum as the pipeline progress line and for tier accents. Less celebratory than the competitor app — dense, legible, fast. Recharts for the small rating/standings charts on `/control/results`.